



Arquitectura de Computadoras

Clase 1: Subrutinas y la Pila

1. Subrutinas (Procedimientos) en Assembler

Una subrutina es un bloque de código que realiza una tarea específica, similar a los procedimientos en Pascal. El programa principal la llama, le puede pasar datos y esta devuelve un resultado.

- **Definición en el Simulador (MSX-88):**
 - **Origen y Ubicación:** Se define en una dirección de memoria **ORIGEN** (comenzando por **3000H** por convención). Esa dirección definida no puede ser **2000H**, ya que esa dirección está reservada siempre para el programa principal. Tampoco debe superponerse con otras zonas del programa.
 - **Estructura:** Comienza con una etiqueta (ej. **MI_SUBROUTINA:**) y, a diferencia del programa principal que termina en **END**, una subrutina siempre finaliza con la instrucción **RET** (retorno).
-

2. Pasaje de Parámetros: Valor vs. Referencia

Cuando una variable se define, se generan dos números importantes: su **contenido** (lo que vale) y su **dirección** (dónde está guardada). El tipo de pasaje depende de cuál de estos dos números se entrega a la subrutina.

- **Por Valor (Copiar el dato)**
 - **Qué se pasa:** Se entrega el **contenido** de la variable.
 - **Consecuencia:** Es un parámetro de **solo entrada**. La subrutina no puede modificar el dato original porque no sabe dónde está guardado.
 -
 - **Por Referencia (Pasar la dirección)**
 - **Qué se pasa:** Se entrega la **dirección** de memoria donde está la variable.
 - **Consecuencia:** La subrutina **sí puede modificar** el dato original, ya que conoce su ubicación exacta.
 - **Eficiencia:** Puede ser menos eficiente para tareas simples (el código se hace más largo), pero es **mucho más eficiente** para pasar grandes estructuras de datos como arreglos, ya que solo se envía la dirección de inicio y no todos los datos.
-

3. ¿Dónde se Pasan los Parámetros? (Medios Físicos)

-  **Registros:** Usar los registros de la CPU (como **AX**). Es simple pero limitado por la poca cantidad de registros disponibles.
-  **Memoria (Variables Globales):** Es como usar variables globales que tanto el programa principal como la subrutina conocen. Es una práctica desaconsejada porque es **difícil de estandarizar**, especialmente cuando diferentes programadores trabajan en el mismo proyecto.

- 🍷 **La Pila (Stack):** Es el método preferido y el que usan la mayoría de los lenguajes de alto nivel. Es una zona de memoria destinada específicamente para este y otros usos.
-

4. La Pila (The Stack) y sus Operaciones

La Pila es una zona de memoria gestionada por el registro **Stack Pointer (SP)**.

- **Ubicación (en el simulador):** El **SP** se inicializa en la dirección fija **8000H**. En un sistema operativo real, esta zona de memoria es asignada automáticamente cada vez que se ejecuta un programa.
- **Operaciones **PUSH** y **POP**:** Siempre trabajan con datos de **16 bits (2 bytes)**.
 - **PUSH AX (Meter en la Pila):** Esta instrucción **NO** escribe en la dirección actual del **SP**. Primero **decrementa el SP en 2** y luego copia el valor de **AX** en esa nueva dirección. La pila "crece" hacia direcciones más bajas.
 - **POP BX (Sacar de la Pila):** Es la operación inversa. Primero **copia el valor** desde donde apunta el **SP** hacia el registro **BX** y luego **incrementa el SP en 2**. El dato no se borra de la memoria, pero queda inaccesible para futuras operaciones de pila. Se puede desapilar en un registro distinto del que se usó para apilar.

5. Pasaje de Parámetros y el Rol Clave del **CALL**

Para pasar datos a una subrutina, la instrucción **CALL** es la que formalmente la invoca. Los parámetros se pueden pasar de dos maneras principales:

- **Vía Registros (El Método Directo):** Se carga el dato en un registro (ej. **move ax, dato**) y luego se llama a la subrutina (**call nombre_subrutina**). Es responsabilidad del programa que llama cargar el dato correctamente.
- **Vía la Pila (El Método Estándar):** Se apila el dato (**push dato**) y luego se llama a la subrutina.

¡El Secreto del **CALL**! 💡 La instrucción **CALL** no solo salta a la subrutina. Automáticamente, **el sistema usa la pila para guardar la dirección de retorno**. Esto significa que **CALL** también modifica el **SP**, apilando 2 bytes adicionales (en el simulador) después de los parámetros que tú hayas apilado.

6. El Gran Desafío: Acceder a los Parámetros Dentro de la Subrutina

Una vez dentro de la subrutina, te enfrentas a un problema: el **SP** ya no apunta a tus parámetros, sino a la **dirección de retorno** que guardó el **CALL**. Por lo tanto, **no se puede usar POP** para obtener los parámetros, ya que sacarías la dirección de retorno y el programa se "rompería".

La Solución: Usar un puntero base estable. En la arquitectura real se usa **BP** (Base Pointer), pero como el simulador no lo tiene, **usaremos el registro BX**.

El protocolo estándar dentro de una subrutina para acceder a los parámetros es:

1. **Salvar el Puntero Base:** Se hace `push bx`. Esto guarda el valor que `BX` pudiera tener en el programa principal, ya que lo vamos a modificar.
 2. **Establecer la Nueva Base:** Se ejecuta `move bx, sp`. Esto copia el valor actual del `SP` a `BX`, creando un "marcador" o "ancla" estable que apunta a la cima de la pila en ese momento.
 3. **Acceder a los Parámetros:** Se accede a los datos usando direccionamiento indirecto con un desplazamiento. Por ejemplo, `[bx + número]`. Se debe sumar un número (`+4`, `+6`, `+8`, etc.) para "saltar" por encima de los datos que no son los parámetros (como el `BX` guardado y la dirección de retorno) y llegar al dato deseado.
 4. **Restaurar antes de Salir:** Justo antes del `RET` final, se debe ejecutar `pop bx` para restaurar el valor original de `BX`, dejando la pila en el estado correcto para que el `RET` funcione.
-
-

Clase 2: El Mundo de las Interrupciones

1. ¿Qué es una Interrupción y Por Qué Existe?

Una interrupción es un mecanismo que permite a los dispositivos de hardware (que son más lentos) **avisarle al procesador que necesitan su atención**. Esto altera la secuencia normal del programa para atender una tarea externa.

- **El Problema a Resolver:** Sincronizar la CPU, que es muy rápida, con periféricos mucho más lentos (teclado, mouse, etc.).
 - **La Alternativa Ineficiente (Polling):** Sin interrupciones, el procesador tendría que estar preguntando constantemente a cada dispositivo: "¿Necesitas algo?", "¿Y tú?", "¿Y tú?". Esto es un gran desperdicio de tiempo y recursos.
-

Tipos de Interrupciones

Existen diferentes "sabores" de interrupciones, dependiendo de quién las genera.

1. Por Hardware

Son las "verdaderas" interrupciones. Son llamadas a subrutinas generadas por una **señal eléctrica externa** proveniente de un dispositivo, como al presionar una tecla o mover el mouse.

- **Asincrónicas:** Ocurren en cualquier momento, sin estar planeadas en el programa.
- **Origen:** Dispositivos de entrada/salida (periféricos).
- **Nombre técnico:** *Interrupt Request*.

El Proceso Detallado de una Interrupción por Hardware

Cuando la CPU acepta una interrupción (siempre al final de un ciclo de instrucción), sigue estos pasos:

1. **Suspender:** Detiene la ejecución del programa actual.

2. **Guardar Contexto:** Guarda en la pila la **dirección de retorno** y el estado de los **flags** para saber exactamente cómo continuar después.
 3. **Cargar Gestor:** Carga la dirección de la "**rutina de gestión de interrupción**" apropiada en el contador de programa.
 4. **Ejecutar y Retornar:** Una vez que el gestor termina, se restaura el contexto y el programa original sigue como si nada hubiera pasado.
-

2. Por Software

A pesar del nombre, **es una instrucción escrita en el programa**. Se usa para llamar a funciones del sistema operativo que no están definidas en nuestro código.

- **Instrucción:** Se utiliza el comando `int` seguido de un número (ej. `int n`).
- **Ejemplos en el Simulador:**
 - `int 6`: Espera a que el usuario presione una tecla (como un `read`).
 - `int 7`: Muestra un mensaje en pantalla (como un `write`).
- **¿Por qué se llama "interrupción"?** Porque el mecanismo que usa la CPU para encontrar y ejecutar la función es el mismo que para las interrupciones por hardware.

¿Por qué usar `int n` en lugar de `CALL`?

La razón es **dónde está el código** que se va a ejecutar.

- Un `CALL` invoca subrutinas que están **definidas dentro de tu propio programa**.
- Una `int n` invoca funciones cuyo código **NO está en tu programa**, como las del **sistema operativo**.

Este mecanismo te permite usar funciones complejas del sistema (como leer del teclado o escribir en pantalla) sin tener que incluir todo su código en tu programa. Es la forma que tiene la máquina de llamar a servicios externos ya cargados en memoria.

3. Gestión de Múltiples Dispositivos: El PIC y su Diálogo con la CPU

El procesador solo tiene una única entrada para interrupciones, pero debe atender a muchos dispositivos. La solución es el **PIC (Controlador de Interrupciones Programable)**. El proceso de comunicación es un diálogo preciso:

1. **El Dispositivo avisa al PIC.** Un dispositivo, ej: El teclado envía una señal a una de las entradas del PIC.
2. **El PIC avisa a la CPU.** El PIC reenvía la señal a la CPU para informarle que hay una interrupción pendiente.
3. **La CPU Acepta.** Cuando está lista, la CPU le responde al PIC que acepta la interrupción.
4. **El PIC identifica al dispositivo.** El PIC sabe quién lo interrumpió, pero la CPU aún no. Para informarle, el PIC envía un "**numerito**" (el número de vector) a la CPU a través del bus de datos.

5. **La CPU busca en el "mapa".** Con ese número, la CPU va a una zona especial de memoria llamada **Tabla de Vectores de Interrupción** para buscar la dirección de la rutina que debe ejecutar.
 6. **La CPU ejecuta la Rutina.** La CPU salta a la dirección que encontró en la tabla y ejecuta el programa de servicio correspondiente.
-

4. La Tabla de Vectores de Interrupción: El "Mapa" de la CPU

Esta tabla es una zona en la parte más baja de la memoria (a partir de la dirección **00**) que contiene las **direcciones de inicio** de todas las rutinas de gestión de interrupciones.

- **Estructura en el Simulador:** Tiene 256 "renglones" o entradas (de 0 a 255), y cada una ocupa 4 bytes.
 - **Ventaja Clave:** Permite una gran flexibilidad. Si se actualiza un driver o una función del sistema operativo, el nuevo programa puede cargarse en cualquier lugar de la memoria. Lo único que se necesita es **actualizar la dirección en la tabla de vectores**; el resto del sistema seguirá funcionando igual.
 - **Unión de Hardware y Software:** Esta tabla es utilizada tanto por las interrupciones de hardware como por las de software. Es por esto que las **int n** se llaman "interrupciones": utilizan el mismo mecanismo de tabla para encontrar la subrutina que deben ejecutar.
-

5. Prioridades y Enmascaramiento: ¿Quién pasa primero?

- **Prioridades:** No todas las entradas del PIC son iguales. La entrada con el **número más bajo tiene la prioridad más alta**. Si dos señales llegan al mismo tiempo, el PIC atenderá primero a la de mayor prioridad. Una interrupción de menor prioridad deberá esperar si una de mayor prioridad está siendo atendida.
 - **Enmascaramiento:** Es posible habilitar o deshabilitar interrupciones de forma individual. A esto se le llama **enmascarar**. Si una interrupción está enmascarada (deshabilitada), el PIC no la atenderá. En el simulador, esto se controla escribiendo en un registro del PIC llamado **"registro máscara"**.
-

6. Las Interrupciones en el Simulador

- **Componentes clave:** el PIC, el Timer, la tecla F10 y el PIO (un dispositivo para conectar LEDs e interruptores).
- **Mapeo de dispositivos:**
 - La tecla **F10** está conectada a la entrada **INT0** del PIC (la de mayor prioridad). Su máscara es el bit 0 del registro máscara.
 - El **Timer** está conectado a la entrada **INT1** del PIC. Su máscara es el bit 1.
- **Vectores ocupados y libres:** De las 256 entradas en la tabla de vectores, 4 ya están usadas por el simulador para interrupciones por software:
 - **INT 0:** Finaliza la ejecución.
 - **INT 3:** Puntos de parada (debugging).
 - **INT 6:** Leer del teclado.

- **INT 7**: Mostrar un cartel en pantalla.

El resto de las entradas están libres para que el programador las utilice.



Clase 3 - El PIC y la Configuración de Interrupciones por Hardware

1. El PIC: El Administrador Central de Interrupciones

Dado que la CPU solo tiene **una única entrada** para recibir interrupciones, no puede gestionar múltiples dispositivos directamente. Aquí es donde interviene el **PIC (Controlador de Interrupciones Programable)**.

- **Función Clave:** El PIC actúa como un "administrador" con múltiples entradas (ocho en el simulador, de **INT0** a **INT7**). Recibe las señales de los distintos dispositivos de hardware y las gestiona para comunicarse ordenadamente con la CPU.
 - **Prioridades:** El PIC asigna prioridades según el número de la entrada: **la entrada con el número más bajo tiene la prioridad más alta**. Si dos señales llegan al mismo tiempo, se atenderá primero a la de mayor prioridad. En el simulador, la tecla **F10 (INT0)** tiene mayor prioridad que el **Timer (INT1)**.
-

2. Conceptos Fundamentales para la Configuración

- **INTn (Hardware) vs. int n (Software):** Es vital no confundirlos. **INT0**, **INT1**, etc., son los **nombres de las entradas físicas** del PIC. En cambio, **int 0**, **int 7**, etc., es la **instrucción de Assembler** para generar una interrupción por software.
 - **Enmascaramiento (Habilitar/Deshabilitar):** Podemos controlar qué interrupciones acepta el PIC a través de su **Registro Máscara (IMR)**. Es un registro de 8 bits donde cada bit corresponde a una entrada (**bit 0** para **INT0**, **bit 1** para **INT1**, etc.).
 - Escribir un **0** en un bit **habilita** la interrupción.
 - Escribir un **1** en un bit la **deshabilita** (la enmascara).
 - **La Instrucción OUT:** Los registros del PIC no son memoria, sino **puertos de entrada/salida**. Por lo tanto, para escribir en ellos no se usa **MOV**, sino la instrucción **OUT**.
-

3. Guía Práctica: Configuración de una Interrupción por Hardware (Ejemplo: F10)

El siguiente proceso detalla cómo configurar el sistema para que la pulsación de la tecla F10 active automáticamente una subrutina.

Paso A: Definir Constantes (**EQU**)

Para hacer el código más legible, se definen constantes:

- `pic EQU 20H`: Se asigna el nombre `pic` a la dirección `20H`, que es la **dirección base** de los registros del PIC.
- `int_num EQU 10`: Se elige un número de vector (`10`) para nuestra rutina. Es un número arbitrario y **no puede ser uno de los ya usados por el sistema (0, 3, 6 y 7)**.

Paso B: El Proceso de Configuración

El orden de estas instrucciones es fundamental para un funcionamiento correcto.

1. **CLI (Deshabilitar Interrupciones a Nivel CPU)**: La primera instrucción es `CLI` (Clear Interrupt Flag). Pone en pausa la atención de interrupciones a nivel del procesador. Es una **medida de seguridad** para evitar que una interrupción llegue mientras estamos a mitad de la configuración.
2. **Configurar el Registro Máscara del PIC (IMR)**:
 - `MOV AL, 0FEH`: Se carga en `AL` el valor `FEH` (`11111110` en binario). Este valor pone un **0 únicamente en el bit 0**, habilitando así la entrada `INT0` (tecla F10) y deshabilitando todas las demás.
 - `OUT PIC+1, AL`: Se envía el valor de `AL` al puerto del `IMR`. La dirección `PIC+1` equivale a `21H`, que es la dirección del `IMR`.
3. **Configurar la Tabla de Vectores**: En este paso (que el profesor explica conceptualmente), se debe escribir la dirección de nuestra subrutina de atención (ej. `3000H`) en la entrada que elegimos (`10`) de la Tabla de Vectores de Interrupción.
4. **STI (Habilitar Interrupciones a Nivel CPU)**: Una vez que todo está configurado, la instrucción `STI` (Set Interrupt Flag) le indica a la CPU que ya puede volver a atender las señales de interrupción que le lleguen desde el PIC.

Tras estos pasos, el sistema queda listo. El programa principal puede estar en un bucle vacío, y cada vez que se presione F10, la subrutina se ejecutará automáticamente.



Clase 4 - Módulos y Técnicas de Entrada/Salida

1. El Problema Fundamental de la Entrada/Salida (E/S)

La comunicación con periféricos es compleja debido a varios desafíos que no existen entre la CPU y la memoria RAM.

- **Diferencias de Velocidad y Formato**: Los periféricos son mucho más lentos que la CPU y manejan diferentes volúmenes y formatos de datos (ej. datos en serie vs. en paralelo).
- **Naturaleza Eléctrica/Mecánica**: A menudo se necesitan interfaces para adaptar las señales eléctricas o mecánicas del periférico a las que maneja la computadora.

La solución a estos problemas es el **Módulo de Entrada/Salida**, un componente intermediario que actúa como traductor.

2. Direccionamiento de E/S: Aislada vs. Mapeada en Memoria

Para que la CPU pueda comunicarse con los periféricos, estos deben tener direcciones. Existen dos estrategias principales para gestionar estas direcciones.

- **E/S Aislada (Isolated I/O)**

- **Concepto:** La memoria y los periféricos tienen **espacios de direcciones separados**. Esto significa que puede existir una dirección de memoria **30H** y, al mismo tiempo, un puerto de E/S con la dirección **30H**.
- **Instrucciones:** La CPU utiliza instrucciones **específicas** para acceder a los puertos de E/S. En la arquitectura Intel (y en el simulador), estas instrucciones son **IN** y **OUT**. La instrucción **MOV** se reserva para la memoria.
- **¿Cómo sabe la CPU cuál usar?** La Unidad de Control decodifica la instrucción. Si es un **MOV**, activa las líneas **Memory Read/Write** del bus de control. Si es un **IN** o **OUT**, activa las líneas **I/O Read/Write**.
- **Desventaja:** Las instrucciones de la ALU (**ADD**, **SUB**, etc.) no pueden operar directamente con los puertos de E/S. Los datos deben ser transferidos primero a un registro de la CPU con la instrucción **IN**.

- **E/S Mapeada en Memoria (Memory-mapped I/O)**

- **Concepto:** La memoria y los periféricos **comparten un único espacio de direcciones**. Las direcciones de los puertos de E/S son tratadas como si fueran direcciones de memoria comunes, "robando" espacio que podría ser usado por la RAM.
- **Instrucciones:** No hay instrucciones especiales. La CPU usa las mismas instrucciones para acceder a la memoria (como **MOV**) para comunicarse con los periféricos.

3. El PIO: Un Dispositivo de E/S Programable en el Simulador

El PIO (Programmable Input-Output) es un dispositivo intermediario que permite conectar periféricos simples como interruptores (llaves) y LEDs a los buses de la CPU.

- **Estructura y Direcciones:** El PIO tiene **cuatro registros de 8 bits**, cada uno en un puerto de E/S aislado:
 - **PA** (Puerto de Datos A): Dirección **30H**.
 - **PB** (Puerto de Datos B): Dirección **31H**.
 - **CA** (Puerto de Control A): Dirección **32H**.
 - **CB** (Puerto de Control B): Dirección **33H**. En el simulador, el Puerto A está conectado a 8 interruptores (entrada) y el Puerto B a 8 LEDs (salida).
- **Modelo de Programación:** El concepto clave es la separación de funciones:
 - Los registros de **Datos (PA, PB)** son por donde la información entra y sale. Aquí es donde se leen los valores de los interruptores o se escriben los valores para encender los LEDs.
 - Los registros de **Control (CA, CB)** se usan para **programar la dirección** de cada bit en los puertos de datos. **CA** configura a **PA** y **CB** configura a **PB**.
 - **La Regla de Configuración:**
 - Para configurar un bit como **salida**, se escribe un **0** en el bit correspondiente del registro de control.
 - Para configurar un bit como **entrada**, se escribe un **1** en el bit correspondiente del registro de control.

- **Ejemplo:** Para leer los 8 interruptores conectados a PA, primero debemos configurar los 8 bits de PA como entrada. Esto se logra escribiendo FFH (ocho 1s en binario) en el registro de control CA usando la instrucción OUT.
-

4. Técnicas de Gestión de E/S: Tres Formas de Trabajar

Existen tres métodos principales para gestionar la transferencia de datos entre la CPU y un módulo de E/S.

1. **E/S Programada (Por Polling):** La CPU tiene el control total. Envía un dato y luego **pregunta constantemente** al módulo si ya terminó. Es ineficiente porque la CPU queda ociosa esperando.
2. **E/S por Interrupciones:** La CPU envía un dato y se dedica a otras tareas. Cuando el periférico está listo, **le avisa a la CPU generando una interrupción**. Es mucho más eficiente que el polling.
3. **Acceso Directo a Memoria (DMA):** Se usa un **controlador DMA** especial. La CPU le ordena transferir un bloque de datos y el DMA lo hace directamente entre la memoria y el periférico, sin involucrar a la CPU hasta el final. Es el método más eficiente para transferir **grandes volúmenes de datos**.